Non-transient `constexpr` allocation

Document #:	P2670R1
Date:	2023-02-03
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Added discussion of `propconst` specifier, as an alternative option to a `propconst` qualifier, and proposing that as the correct choice instead.

§ 2 Introduction

For C++20, [P0784R7] introduced the notion of transient `constexpr` allocation. That is, we can allocate during compile time - but only as long as the allocation is completely cleaned up by the end of the evaluation. With that, we can do this:

```
constexpr auto f() -> int {
    std::vector<int> v = {1, 2, 3};
    return v.size();
}
static_assert(f() == 3);
```

But not yet this:

```
constexpr std::vector<int> v = {1, 2, 3};
static_assert(v.size() == 3);
```

Because `v`'s allocation persists, that is not yet allowed.

Similarly, whether this is valid or not depends entirely on what small string optimization implementation strategy a standard library implementation took years ago:

```
constexpr std::string s = "hello";
```

§ 3 The Problem Case

The problem with persistent `constexpr` allocation can be demonstrated with this example:

```
1  // simplified version of unique_ptr
2  template <class T>
3  class unique_ptr {
4      T* ptr;
5  public:
6      explicit constexpr unique_ptr(T* p) : ptr(p) { }
7      constexpr ~unique_ptr() { delete ptr; }
8
9      // unique_ptr is shallow-const
10     constexpr auto operator*() const -> T& { return *ptr; }
11     constexpr auto operator->() const -> T* { return ptr; }
12
13     constexpr void reset(T* p) {
14         delete ptr;
15         ptr = p;
16     }
17 };
18
19 constexpr unique_ptr<unique_ptr<int>> ppi(new unique_ptr<int>(new int(1)));
20
21 int main() {
22     // this call would be a compile error, because ppi is a const
23     // object, so we cannot call reset() on it
24     // ppi.reset(new unique_ptr<int>(new int(2)));
25
26     // but this call would compile fine
27     ppi->reset(new int(3));
28 }
```

In order for constexpr allocation to persist to runtime, we first need to ensure that all of the allocation is cleaned up properly. We can check that our allocation (stored in `ppi.ptr`) is properly deallocated in its destructor (which it is). And so on , recursively - so `ppi.ptr->ptr` is properly cleaned up in `*(ppi.ptr)`'s destructor.

Once we verify that constexpr destruction properly deallocates all of the memory, no actual destructor is run during runtime. In order for this to be valid, it is important that the destruction that would happen at run-time is actually the same as the destruction that was synthesized during compile-time.

In the above example though, that is not the case.

`ppi` is a `unique_ptr<unique_ptr<int>> const`, so its member `ppi.ptr` is a `unique_ptr<int>* const`. The pointer itself is `const` (top-level), which means that we cannot change what pointer it owns. This means that destroying `ppi` at runtime would definitely `delete` the same pointer that it was instantiated with at compile time (short of `const_cast` shenanigans, which we can't do much about and are clearly UB anyway). So far so good.

But `*ppi` gives me a *mutable* `unique_ptr<int>`, which means I can change it. In particular, while `ppi.reset(~)` would be invalid, `(*ppi).reset(~)` would compile fine. Our tentative evaluation of the destructor of `*(ppi.ptr)` would try to delete the pointer which was initialized with `new int(1)`, but now it would point to `new int(3)`. That's something different, which means that our synthetic destructor evaluation was meaningless.

More to the point - the `delete ptr;` call (on line 14) would end up attempting to delete memory that wasn't allocated at runtime - it was allocated at compile time and promoted to static storage. That's going to fail at runtime. This is highly problematic: the above use looks like perfectly valid C++ code, and we got into this problem without any `const_cast` shenanigans or really even doing anything especially weird. It's not like we're digging ourselves a hole, it's more that we just took a step and fell.

§ 3.1 Mutable Allocation

The problem isn't that the allocation is mutable. It's much more subtle than that. There are no problems allowing this:

```
constexpr unique_ptr<int> pi(new int(4));
```

While the data `pi` points to is mutable, it can be mutated at runtime without any issue. The important part of this is that the allocation is pointed to by a const pointer (`pi.ptr` here is an `int* const`), so there is no way to change *which* pointer is the allocation. Just what its data is.

The expectation would be that the above program would basically be translated into something like:

```
int __backing_storage = 4;
constexpr int* pi = &__backing_storage;
```

That is perfectly valid code today - it's the value of the pointer that is a constant, not what it is pointing to.

§ 3.2 Mutable Reads During Constant Destruction

The problem is specifically when a *mutable* object is *read* during *constant destruction*.

In the previous section, `pi.ptr` (an `int* const`) is read during constant destruction, but it is not mutable . `*(pi.ptr)` is mutable (an `int`), but wasn't read.

In the original example, `ppi.ptr` (a `unique_ptr<int>* const`) is read during constant destruction , but it is not mutable. But `ppi.ptr->ptr` *is* mutable (it's just an `int*`) and was read during `*(ppi.ptr)`'s constant destruction, which is the problem.

While `constexpr unique_ptr<unique_ptr<T>>` might not seem like a particularly interesting example, consider instead the case of `constexpr vector<vector<T>>` (or, generally speaking, any kind of nested containers - like a `vector<string>`). Both `unique_ptr<T>` and `vector<T>` have a `T*` member, so they're structurally similar - it's just that `unique_ptr<T>` has a shallow const API while `vector<T>` has a deep const API.

But this is purely a convention. How can the compiler distinguish between the two? How can we disallow the bad `T*` case (`unique_ptr<unique_ptr<T>>` or `unique_ptr<string>` or `unique_ptr<vector<T>>`) while allowing the good `T*` case (`vector<vector<T>>` or `vector<string>`)?

§ 4 Proposed Solutions

There have been two proposed solutions to this problem, both of which had the goal of rejecting the nested `unique_ptr` case:

```
constexpr unique_ptr<int> pi(new int(1));                                  // ok
constexpr unique_ptr<unique_ptr<int>> ppi(new unique_ptr<int>(new int(2))); // error
```

While allowing the `vector` cases:

```
constexpr std::vector<int> vi = {1, 2, 3};                              // ok
constexpr std::vector<std::vector<int>> vvi = {{1, 2}, {3 4}, {5, 6}}; // ok
```

```
constexpr std::vector<std::string>> vs = {"this", "should", "work"};   // ok
```

§ 4.1 `std::mark_immutable_if_constexpr`

The initial design in [P0784R7], before it was removed, was a new function `std::mark_immutable_if_constexpr(p)` that would mark an allocation as being immutable. That is, rather than rejecting any object that is mutable and read during constant destruction, the new rule would be any object read during constant destruction must be either:

— constant, or

— marked as immutable

`std::vector<T>` would mark its allocation as immutable (e.g. at the end of its constructor), but `std::unique_ptr<T>` would not (because it's only shallow const).

That way:

```
// ok: this is fine (no mutable object is read during constant destruction)
constexpr std::unique_ptr<int> a(new int(1));

// error: allocation isn't marked immutable, but is read as mutable during constant
        destruction
constexpr std::unique_ptr<std::unique_ptr<int>> b(new std::unique_ptr<int>(new int(2)));

// ok: allocation isn't marked immutable, but is read as constant
constexpr std::unique_ptr<std::unique_ptr<int> const> c(new std::unique_ptr<int>(new
        int(3)));

// ok: allocation isn't read as mutable
constexpr std::vector<int> v = {3, 4, 5};

// ok: allocation is read as mutable but is marked immutable
constexpr std::vector<std::vector<int>> vv = {{6}, {7}, {8}};
```

In this way, `std::mark_immutable_as_constexpr(p)` is basically saying: I promise my type is actually properly deep-`const`. Note that there's no real enforcement mechanism here - if the user did mark their `unique_ptr` member immutable, then the original example in this paper would compile fine (and fail at runtime). It's just a promise to the compiler that your type is actually deep-`const`, it's up to the user to properly ensure that it is.

§ 4.2 `T propconst*`

[P1974R0] proposed something different: a new qualifier that would propagate `const`-ness off the object through the pointer. A persistent `constexpr` allocation would thus have to have only `const` access, after adjusting through `propconst`.

`std::vector<T>` would change to have members of type `T propconst*` instead of `T*`, so that a `constexpr std::vector<int>` would effectively have members of type `int const*`, while `std::unique_ptr<T>` would remain unchanged, still having a `T*`. That way:

```
// ok: this is fine (no mutable object is read during constant destruction)
constexpr std::unique_ptr<int> a(new int(1));

// error: b's pointer member has mutable access to a std::unique_ptr<int>
// which is a mutable read during constant destruction
constexpr std::unique_ptr<std::unique_ptr<int>> b(new std::unique_ptr<int>(new int(2)));

// ok: each allocation is read as constant
constexpr std::unique_ptr<std::unique_ptr<int> const> c(new std::unique_ptr<int>(new
        int(3)));
```

```
// ok: allocation isn't read as mutable
constexpr std::vector<int> v = {3, 4, 5};

// ok: allocation isn't read as mutable, because the member is now a vector<int>
        propconst* rather
// than a vector<int>*, so behaves as if it were a vector<int> const*
constexpr std::vector<std::vector<int>> vv = {{6}, {7}, {8}};
```

The distinction between the two proposals for a simple `vector` implementation:

`mark_immutable_if_constexpr`

```
template <typename T>
class vector {
    T* ptr_;
    size_t size_;
    size_t capacity_;

public:
    constexpr vector(std::initializer_list<T>
        elems) {
        ptr_ = new T[xs.size()];
        size_ = xs.size();
        capacity_ = xs.size();
        std::copy(xs.begin(), xs.end(), ptr_);

        std::mark_immutable_if_constexpr(ptr_);
        // <==
    }

    constexpr ~vector() {
        delete [] ptr_;
    }
}
```

`T propconst*`

```
template <typename T>
class vector {
    T propconst* ptr_; // <==
    size_t size_;
    size_t capacity_;

public:
    constexpr
        vector(std::initializer_list<T>
        elems) {
        ptr_ = new T[xs.size()];
        size_ = xs.size();
        capacity_ = xs.size();
        std::copy(xs.begin(), xs.end(),
        ptr_);
    }



    constexpr ~vector() {
        delete [] ptr_;
    }
}
```

On the left, we have to mark the result of every allocation immutable. On the right, we have to declare every member that refers to an allocation with this new qualifier.

§ 4.3 A `propconst` specifier

The previous section was what Jeff Snyder actually proposed: a `propconst` *qualifier*, to be used in the same position as `const` is used today. An alternative approach, suggested to me recently by Matt Calabrese, would be instead to have a `propconst` *specifier* - used in the same way that the `mutable` keyword is used today. The question is: how would such a specifier behave?

With the `propconst` qualifier, we can choose at which level(s) to add the qualifier if we have a multi-layered pointer type. With the `propconst` specifier, the language has to make a singular choice for all contexts. For instance, the user can write `int propconst**` or `int propconst* propconst*` or `int* propconst*` if `propconst` is a qualifier. But what should `propconst int** p;` mean as a declaration, when `p` is accessed as `const`? It could mean:

— `int const* const*` (`const` at every level)

— `int const**` (`const` only at inner level)

— `int* const*` (`const` only at outer level)

Immediately we can reject the choice of `int const**`. An `int const**` is actually not convertible to `int**`, since such a conversion would open a whole in the type system allowing you to inadvertently modify a const object (see the example in 7.3.6 [conv.qual]/3). That reduces the choice to either every level or outer-only.

Consider the use-case of `vector<T>` (as pointed out by Tim Song). `vector<T>::data() const` returns a `T const*`, for all `T`. If `T` is, itself, a pointer type, then `vector<int*>::data() const` returns an `int* const*`. It's only one level of `const`-ness: the user cannot mutate the pointers themselves, but they can mutate through the pointer. If `propconst` applied `const` at every level, then implementing `vector<T>` using a `propconst T* begin_` declaration would end up giving us an `int const* const*` instead, which cannot be converted to the `int* const*` that we need. The consequence of this is basically that `vector<U*> const` becomes unusable as a type, even outside of `constexpr`. That's clearly unacceptable.

That suggests that the answer is: `const` only at the outer level. `propconst int** p;` behaves like an `int* const*`, `propconst int*** q;` behaves like an `int** const*`, etc.

Now, consider the use-case of a matrix class implemented using layers of pointers (not the ideal implementation):

2D Matrix

```
struct Matrix2D {
    propconst int** p;
    int n;

    constexpr ~Matrix2D() {
        for (int i = 0; i != n; ++i) {
            delete [] p[i];
        }
        delete [] p;
    }
};
```

3D Matrix

```
struct Matrix3D {
    propconst int*** p;
    int n;

    constexpr ~Matrix3D() {
        for (int i = 0; i != n; ++i) {
            for (int j = 0; j != n; ++j) {
                delete [] p[i][j];
            }
            delete [] p[i];
        }
        delete [] p;
    }
};
```

Assuming that `propconst` adds `const` only at the outer layer, do these types work with non-transient constexpr allocation? `Matrix2D` does, but `Matrix3D` does not. We get *one* layer of added `const`-ness, so `Matrix2D::p` is treated as an `int* const*`. That means that you can mutate the underlying values (you can change `p[0][0] = 42;`), but those values aren't read in the destructor, so their mutation doesn't matter. What you can't mutate are any of the pointers, so this is fine. Similarly, in `Matrix3D`, we get *one* layer of added `const`-ness, so `Matrix3D::p` is treated as an `int** const*`. While you can't mutate `p`, or any of the pointers `p[i]`, you now *can* mutate the pointers the next layer down (e.g. `p[0][0] = new int(42);`). Because that mutation isn't protected, this allocation can't be allowed.

In this case, we don't want *just* the outer layer, we actually want all the layers (although we could make do with all but the inner-most one). What if we added a new option "all but inner (if more than one)"?

declaration	outer only	all but inner (if more than one)
`propconst int*`	`int const*`	`int const*`
`propconst int**`	`int* const*`	`int* const*`
`propconst int***`	`int** const*`	`int* const* const*`
`propconst int****`	`int*** const*`	`int* const* const* const*`

But we already know that the right-most column breaks for `std::vector<T>`: if we had a `std::vector<int**>`, we need to produce an `int** const*` (as described earlier), not an `int* const* const*` (as would occur in the right-most column).

There simply isn't one correct choice for all use-cases, which is kind of a problem with trying to solve this case with a facility (specifier) that requires picking just one.

But we could actually have our cake and eat it too here: we could have a `propconst` specifier, but also specify how many layers of `const` we're adding, where by default it applies to every layer:

declaration	meaning
`propconst int`	`int`
`propconst int*`	`int const*`
`propconst(1) int*`	`int const*`
`propconst(2) int*`	`int const*`
`propconst int**`	`int const* const*`
`propconst(1) int**`	`int* const*`
`propconst(2) int**`	`int const* const*`
`propconst(3) int**`	`int const* const*`
`propconst int***`	`int const* const* const*`
`propconst(1) int***`	`int** const*`
`propconst(2) int***`	`int* const* const*`
`propconst(3) int***`	`int const* const* const*`

With that rule, `vector<T>` would use `propconst(1)` (since that is the only option: it must add exactly one layer of `const`) while the `N`-dimension `Matrix` class would probably use either `propconst` or `propconst(N)` (it could potentially use `propconst(N-1)`, but that probably doesn't make sense for that use-case).

Note that it's important that `propconst int` and `propconst(2) int*` aren't ill-formed because of use in dependent contexts. In reality, it's not `vector<T>` that has a `T*` member, it's `vector<T, A>` that has an `allocator_traits<A>::pointer` member, where that `pointer` type may not be a language pointer. If we have a `propconst` specifier, that specifier can only apply to this member, so `propconst(1) pointer begin_;` needs be valid (and just do nothing) if `begin_` happens to be a fancy pointer.

Here is a comparison of the difference in `vector<T>` implementations, both the simplified and allocator versions (note that `propconst(1)` is necessary here to ensure that we only ever add one layer of `const` if `T` is, itself, a pointer type):

Qualifier (P1947R0)	Specifier

`template <typename T>`
`struct vector {`
`  T propconst* begin_;`
`  T propconst* end_;`
`  T propconst* capactity_;`
`};`
	`template <typename T>`
	`struct vector {`
	`  propconst(1) T* begin_;`
	`  propconst(1) T* end_;`
	`  propconst(1) T* capactity_;`
	`};`

Or with allocators (where with a qualifier we need to use some kind of trait, because the `propconst` has to go inside the `*`):

Qualifier (P1947R0)	Specifier

`template <typename T>`
`struct propagating_type {`
`  using type = T;`
`};`
 	
`template <typename T>`
	`template <typename T>`
	`struct vector {`
	`  using pointer =`
	`  allocator_traits<Alloc>::pointer;`
	
	`  propconst(1) pointer begin_;`

Qualifier (P1947R0)	Specifier

`struct propagating_type<T*> {`
`  using type = T propconst*;`
`};`
 	
`template <typename T>`
`using propagating = propagating_type<T>::type;`
 	
`template <typename T, typename Alloc>`
`struct vector {`
`  using pointer =`
`  propagating<allocator_traits<Alloc>::pointer>;`
 	
`  pointer begin_;`
`  pointer end_;`
`  pointer capactity_;`
`};`
	`  propconst(1) pointer end_;`
	`  propconst(1) pointer capactity_;`
	`};`

§ 4.4 `propconst` qualifier vs `propconst` specifier

Both approaches are similar - ultimately the goal is to have a member whose type is `T*` but, when the object is const, behave as if it were `T const*`, so that the compiler can ensure no mutable access.

Having a `propconst` qualifier means it's in the type system. This is fairly pervasive through the language and library. Have a `propconst` specifier limits the scope pretty dramatically.

One of the interesting aspects of `propconst` as a qualifier is having language facility that can propagate const through pointers and references: it is possible to make `unique_ptr<int propconst>` give you a `int*` or `int const*` based on the const-ness of the `unique_ptr` object. But in order to do so, we'd have to change the implementation to be:

```
  template <typename T>
  struct unique_ptr {
-   auto get() const -> T*;
+   auto get() -> T*;
+   auto get() const -> T* const;
  };
```

This is discussed in section 7 of the paper ("Extension: function return types"). The extension here is to allow `T* const` as the return type, which is otherwise a fairly silly thing to do, but in this case if `T` is `U propconst` for some `U`, would become `U const*`. This is possible to write with a type trait, but proper language support seems much better to me.

A similar approach could make `std::tuple<Ts propconst...>` a const-propagating tuple if any of the `Ts` are pointers or references. Well, not *exactly* that since we need to have types like `T propconst*` and not `T* propconst`, so this would have to be a type trait of some sort. But given that type trait, we could change the overload of `std::get` on a `tuple<Us...> const&` to return `tuple_element<I, tuple> const& const`. Or something to that effect.

That benefit wouldn't be possible with a `propconst` specifier, since it wouldn't allow you to write `unique_ptr<int propconst>` or `tuple<int propconst*>` to begin with.

But it's not clear how much of a benefit writing `unique_ptr<int propconst>` really is, given that there's a bunch of library work necessary to even take advantage of this possibility, not to mention all the other language complexity to consider - especially when it is fairly straightforward to write `propagate_const<unique_ptr<int>>` if that is really desired.

The specifier approach seems to give you really the bulk of the benefit of the facility (the ability to permit non-transient constexpr allocation) with a fairly minor loss (the ability to declare a const-propagating `unique_ptr` or `tuple`) with

significantly less complexity.

§ 4.5 Disposition

All of these proposals have similar shape, in that they attempt to reject the same bad thing (e.g. `constexpr std::unique_ptr<std::unique_ptr<int>>`) by way of having to add annotations to the good thing (allowing `constexpr std::vector<std::vector<T>>` to work, by either annotating the pointers or the allocation). Same end goal of which sets of programs would be allowed.

Importantly, both require types opt-in, somehow, to persistent allocation. The approaches with `propconst` are more sound, in that the compiler *ensures* that there is no mutable persistent allocation possible, while `std::mark_immutable_if_constexpr(p)` is simply a promise that the user did their due diligence and properly made their type deep-`const`.

What I mean is that, using `propconst` (in either form) this error won't compile - we can't accidentally expose mutable access (not without `const_cast`):

```
template <typename T>
class bad_vector_pc {
    T propconst* data_;
    int size_;
    int capacity_;

public:
    // oops, int& instead of int const&
    constexpr auto operator[](int idx) const -> T& {
        // error: data_[idx] is an T const& here
        return data_[idx];
    }
};
```

But this would work just fine (until it explodes at runtime):

```
template <typename T>
class bad_vector_miic {
    T* data_;
    int size_;
    int capacity_;

public:
    constexpr bad_vector_miic(std::initializer_list<int> xs) {
        data_ = new T[xs.size()];
        std::copy(xs.begin(), xs.end(), data_);
        size_ = xs.size();
        capacity = xs.size();

        std::mark_immutable_if_constexpr(data_);
    }

    constexpr ~bad_vector_miic() {
        delete [] data_;
    }

    // oops, T& instead of T const&
    constexpr auto operator[](int idx) const -> T& {
        // this still compiles though
        return data_[idx];
    }
};
```

```
constexpr bad_vector_miic<bad_vector_miic<int>> v = {{1}, {2}, {3}};

int main() {
    v[0] = {4, 5}; // ok: compiles?
}
```

The `propconst` qualifier approach has the downside that it's fairly pervasive throughout the language - where a significant amount of library machinery would have to account for it somehow. Whereas the `propconst` specifier approach is extremely limited, and `std::mark_immutable_if_constexpr(p)` is even more so. No language machinery needs to consider their existence, and the only library machinery that does are those deep-const types that perform allocation that would have to use it, in only a few places.

On the other hand, `std::mark_immutable_if_constexpr` has the problem that users might not understand where and why to use it and, just as importantly, when *not* to use it. If some marks an allocation immutable on a shallow-const type, that more or less defeats the entire purpose of the exercise, since now they've just allowed themselves to do exactly the thing that wanted to avoid allowing.

A previous revision had suggested adding a `propconst` *class* annotation, that would apply to all of the class's members - effectively like the `propconst` *variable* annotation Matt had suggested here. Having a `propconst` variable specifier strikes me as superior - the annotation belongs on the variables, and having it at that point seems like the most valuable spot for it.

4.6 Alternatives

An entirely different approach would be to eschew annotations altogether. That is - just allow all of these examples to compile, and make clear that it's undefined behavior to mutate persistent constexpr allocations (the allocation itself, not what's written in it) if they're read by the elided destructor. One downside of `std::mark_immutable_if_constexpr(p)` is that users might just eagerly add it to all allocations, even if they shouldn't - and if such a thing happens, then why even bother adding such a function? It's hard to say how often such a facility would be misused - and at least it's not overly difficult to provide good guidance for exactly when one should use it: on types that own allocations such that a constant object only provides constant access through that allocation. Perhaps a longer name like `std::allocation_is_deep_constant(p)` might convey this better, or `std::i_solemnly_swear_that_i_am_up_to_no_mutation(p)`. But also, having a facility such as `std::mark_immutable_if_constexpr(p)` gives users of correctly-written libraries (like `std` and `boost`) protection against accidentally writing `constexpr unique_ptr<string>` or `constexpr unique_ptr<vector<T>>`.

I'm not sure this is better than the three options on the table, but it's at least worth mentioning.

5 Proposal

Between `std::mark_immutable_if_constexpr(p)`, `T propconst*` (the qualifier), and `propconst(N) T*` (the specifier), the latter two provide a sound solution to the problem with neither false positives nor false negatives. The magic function is unsafe and, like Rust's `unsafe`, needs to be carefully reviewed, and can easily provide false negatives (accepted code that really should've been rejected). But, like Rust's `unsafe`, it should only appear in a very small number of places anyway, and making sure those uses are correct doesn't seem like an outrageous burden. After all, of all the types that own an allocation - there are probably way more containers (deep const) than smart pointers (shallow const),[1] so a random use is more likely to be correct than not.

In [P2670R0], I had argued that that `std::mark_immutable_if_constexpr(p)` was a better approach than `T propconst*` (the qualifier) on the basis of the complexity of the latter and the ultimately limited use of the former. But with the introduction of the `propconst(N) T*` (the specifier) idea, I think it might be the right one: it's a sound solution to the problem, that is still limited in scope as far as language and library creep is concerned, while also being easier to explain and understand: we need to ensure that this const object's allocation only has const access, and we need to

propagate const to ensure that is the case. I think that's more straightforward to understand than `std::mark_immutable_if_constexpr` and, importantly, it's also safer: the facility can ensure correctness.

Thus, I'm proposing that the right approach to solving the non-transient constexpr allocation problem is modify [P1974R0] by changing `propconst` from a type qualifier to a storage class specifier, with two forms: `propconst` and `propconst(N)`. The former adds `const` at every layer of pointer/reference-ness, while the latter adds `const` only at the *outer* `N` layers. But otherwise, with the same requirements on when non-transient allocation is allowed to persist.

§ 6 References

[P0784R7] Daveed Vandevoorde, Peter Dimov,Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.
https://wg21.link/p0784r7

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.
https://wg21.link/p1974r0

[P2670R0] Barry Revzin. 2022-10-15. Non-transient constexpr allocation.
https://wg21.link/p2670r0

---

1. This might suggest that we should mark shallow const allocations as shallow const, rather than marking deep const allocations as deep const. The problem is that this requires users to take active action to reject bad code, rather than active action to allow good code – which makes it more likely that the bad code will persist. On the other hand, how many smart pointers are there really?↩